LLM-Driven Iterative Software Testcase Generation Tool

1. 项目目标与研究问题

本项目围绕一个 LLM 驱动的自动化测试生成工具展开，其中 `improved agent` 是本文的主要研究对象，`naive agent` 作为对照基线用于提供参考比较。
其中，`naive agent` 代表保守的一轮生成方案，用于回答一个基本问题：在不引入更强 prompt 设计与覆盖率驱动迭代的情况下，测试生成可以达到怎样的基线水平。

在此基础上，本项目真正关注的是：
通过更强的 prompt 设计与白盒迭代机制，`improved agent` 能否在真实开源项目上生成更高质量的测试用例。

基于上述设定，本文围绕以下研究问题展开：

1. 在真实开源项目上，LLM 能否生成可执行、可用的测试用例？
2. 在 `blackbox`、`whitebox`、`whitebox_code_only` 三种输入模式下，测试质量如何变化？
3. 相较只作为 baseline 的 `naive agent`，`improved agent` 的 prompt 增强与 iteration 机制到底提升了什么？
4. 在 `improved whitebox` 中，哪些设计是有效改进，哪些设计是错误方向？

2. 系统设计、输入与输出

本节重点介绍本文的方法主体 `improved agent`。`naive agent` 仅作为对照基线，不作为系统设计重点展开。

2.1 总体流程

系统整体流程如下：

1. 读取目标函数的 requirement 和/或 source code。
2. 根据 mode 构造 prompt，请 LLM 生成结构化测试用例。
3. 将输出规范化为统一 JSON，包含 `obligations` 与 `test_groups`。
4. 在真实实现上执行测试，得到 correctness。
5. 收集代码覆盖率，得到 `Stmt / Branch / Func / Line` 四项指标。
6. 与人工黄金测试做语义类别对齐，得到 `matched_golden_ratio`。
7. 若为 `improved` 白盒模式，则继续利用 coverage feedback 做 patch 迭代。

2.2 输入

系统输入主要包括 `requirement` 与 `project code base`。
项目支持三种 mode：

- `blackbox`：只提供 requirement。
- `whitebox`：同时提供 requirement 与 source code。
- `whitebox_code_only`：只提供 source code。

对于 `improved` 白盒迭代，每轮额外输入：

- 当前完整 suite；
- `coverage_total`；
- `coverage_files`；
- `uncovered_details`。

2.3 输出

系统输出为严格 JSON 形式的 `test cases`。
统一格式如下：

- 顶层键：
  - `obligations`
  - `test_groups`
- 每个 `test_group` 包含：
  - `title`
  - `validator`
  - `args`
  - `valid`
  - `invalid`
  - `rationale`
  - `obligations`

2.4 `naive agent`：对照基线

`naive agent` 的设计目标是提供一个保守的第一轮对照基线。

其特点是：

- 只生成第一版“小而稳”的 suite；
- 不主动追求 completeness；
- 不做深入 branch hunting；
- 更偏 common cases、obvious boundaries；
- 白盒下也只把代码当作“发现 obvious branches / guards”的辅助信息。

因此，`naive agent` 可以概括为：

保守、低风险、偏 correctness 的基线生成方案。

2.5 `improved agent`：本文方法主体

`improved agent` 的核心设计有两个重点：

1. **Prompt 增强**
2. **白盒 iteration**

2.5.1 `blackbox` 模式

在 `blackbox` 模式下，`improved agent` 不只是“从 requirement 写几个 case”，而是显式要求模型：

- 做 equivalence partitioning；
- 做 boundary value analysis；
- 做 option-combination testing；
- 先提 obligations，再做 category clustering，再展开 test groups；
- 最后做 self-audit，检查 rare options、边界和 option flip 是否遗漏。

相比之下，`naive blackbox` 只做保守的一轮基线生成。
因此，`improved blackbox` 的优势主要来自**更强的 prompt 结构化设计**，而不是 iteration。

2.5.2 `whitebox` 模式

在 `whitebox` 模式下，`improved agent` 是整个项目最核心的研发对象。

其设计包括两层：

1. **静态 prompt 增强**
    - 明确要求 statement coverage、branch coverage、condition-oriented reasoning；
    - 同时关注 requirement-visible behavior 与 code-visible structure；
    - 引导模型枚举 structural white-box obligations；
    - 引导其关注 early return、helper-sensitive path、boundary-triggered branches。
2. **动态覆盖率迭代**
    - 每一轮都提供当前完整 suite；
    - 提供 `requirement`、`coverage detail`、`uncovered_details`；
    - 模型只输出 patch groups；
    - 系统做 `current suite + patch` 得到下一轮 suite。

因此，`improved whitebox` 的价值不只是“看了代码”，而是：

先通过更强的 white-box prompt 做结构化初始生成，再通过 coverage-guided patch iteration 逐轮补路径。

2.5.3 `whitebox_code_only` 模式

在 `whitebox_code_only` 模式下，系统不再提供 requirement，只给 source code。

`improved agent` 会：

- 明确把任务界定为 code-only white-box testing；
- 只从 code-visible contracts 中保守推断 externally visible behavior；
- 强调不应做缺乏代码支持的 speculative claims；
- 在迭代时只使用完整 suite、coverage detail 与 uncovered details。

这个模式的研究意义是：

- 检验“只有代码时，LLM 能否做出接近白盒测试的决策”；
- 暴露 requirement 缺失时的语义漂移风险。

2.6 为什么 `improved` 的优势主要来自 prompt + iteration

`improved` 的性能提升，不应被简单理解为“模型更强”。
更准确地说，它来自两类工程设计：

1. **Prompt 层面的增强**
   - 在 blackbox 中提升语义类别展开能力；
   - 在 whitebox 中提升路径意识与结构意识；
   - 在 code-only 中限制无依据推断。
2. **Iteration 层面的增强**
   - 在白盒场景下利用 coverage / uncovered details 做 patch refinement；
   - 通过 `suite + patch` 保留已有有效 groups，避免整套重写带来的破坏。

因此，`improved agent` 构成了本文的主要方法设计；`naive agent` 的作用则是提供稳定的对照基线。

3. 选择 `validator.js` 作为测试库的原因

选择 `validator.js` 作为实验对象，主要有以下原因：

1. 它是一个真实、成熟、被广泛使用的开源项目，而不是人工构造的 toy benchmark。
2. 它同时具备 requirement、source code 与人类专家编写的测试集，适合进行黑盒与白盒联合评估。
3. 它的函数接口具有一定多样性，既有相对简单的 validator，也有带有选项参数、边界条件和复杂分支的 validator。

更重要的是，`validator.js` 同时提供了两种 ground truth：

- **真实实现本身**：用于判断生成测试是否执行正确；
- **人工黄金测试集**：用于从等价类/语义类别角度评估生成测试与专家测试的接近程度。

因此，它非常适合本项目同时评估：

- correctness；
- coverage；
- semantic overlap。

4. 选择的三类核心评价指标

4.1 正确率 `exact_match_rate`

含义：生成测试在真实实现上执行后，判断正确的比例。

它反映的是：

- 测试是否能真正执行；
- 测试断言是否正确；
- LLM 是否正确理解了函数的外部行为。

选择原因：
如果测试断言写错，即使覆盖率很高，也不能视作高质量测试。

4.2 覆盖率 `Stmt / Branch / Func / Line`

包括：

- `Stmt Cov`
- `Branch Cov`
- `Func Cov`
- `Line Cov`

它反映的是：

- 测试是否触达了更多实现路径；
- 白盒信息是否真正转化为了路径探索能力；
- 迭代 patch 是否命中了之前未覆盖的代码位置。

4.3 黄金测试重合率 `matched_golden_ratio`

含义：生成测试与人工黄金测试在语义类别上的重合比例。

它反映的是：

- 是否覆盖了主要等价类；
- 是否接近专家测试设计的语义结构；
- 生成测试是否只是“碰巧执行正确”，还是在测试设计上也具有合理性。

4.4 为什么必须同时看三类指标

本项目最重要的发现之一是：

correctness、coverage、golden overlap 三者相关，但并不等价。

例如在 `isCurrency` 上，经常出现：

- coverage 很高，甚至达到 `100%`；
- golden overlap 也很高；
- 但 correctness 仍明显偏低。

这说明：

- 走到代码路径，不等于断言就对；
- 与专家测试类别相似，不等于每个 case 都正确；
- 所以实验不能只看 coverage，也不能只看 correctness。

5. 实验设计

最终用于结论的函数集合为：

- `isEmail`
- `isURL`
- `isFQDN`
- `isCurrency`

- `isCreditCard`

总实验数为：

- `5` 个函数
- `3` 种 mode
- `2` 类 agent

即总计 `30` 条实验。

其中：

- `naive` 作为 baseline；
- `improved` 作为项目主方法；
- 比较三种 mode：
  - `blackbox`
  - `whitebox`
  - `whitebox_code_only`

之所以删除 `isUUID` 以及更早删除一些“太简单”或“golden 提取不稳定”的函数，是因为实践中发现：

- 太简单的函数会让所有方法都表现很好，稀释真正困难函数上的差异；
- ground truth 不稳定的函数会破坏结论解释；
- 高质量 benchmark 需要“有区分度 + 有可靠 ground truth”。

6. 实验结果

6.1 总体结果

最终实验结果来自 `experiment_final_1.md` 的完整数据：

- 实验总数：`30`
- 成功实验：`30`
- 总 LLM 请求数：`140`
- 总 token 消耗：`915,473`
- 平均正确率：`0.8599`
- 平均黄金测试重合率：`0.7915`
- 平均语句覆盖率：`86.52`
- 平均分支覆盖率：`80.21`
- 平均函数覆盖率：`100.00`
- 平均行覆盖率：`86.55`

6.2 按 `mode + approach` 汇总

Agent	Mode	Avg Tokens	Avg Exact Match	Avg Stmt Cov	Avg Branch Cov	Avg Func Cov	Avg Line Cov	Avg Golden Match
`naive`	`blackbox`	11259.80	0.8826	81.78	74.89	100.00	81.72	0.5942

Agent	Mode	Avg Tokens	Avg Exact Match	Avg Stmt Cov	Avg Branch Cov	Avg Func Cov	Avg Line Cov	Avg Golden Match
`naive`	`whitebox`	19494.40	0.9167	84.27	78.74	100.00	84.33	0.7714
`naive`	`whitebox_code_only`	18603.80	0.8855	83.71	75.11	100.00	84.49	0.5002
`improved`	`blackbox`	21537.20	0.8823	83.98	78.53	100.00	84.09	0.9667
`improved`	`whitebox`	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857
`improved`	`whitebox_code_only`	53934.60	0.7793	90.19	81.70	100.00	89.66	0.9312

6.3 完整结果矩阵

Validator	Agent	Mode	Status	Correct/Total	Exact Match Rate	Stmt Cov	Branch Cov	Func Cov	Line Cov	Matched Golden Ratio	Total Tokens	Note
isEmail	naive	blackbox	success	44/46	0.9565	73.81	68.42	100.00	73.75	0.5000	12093	ok
isEmail	naive	whitebox	success	41/42	0.9762	73.81	68.42	100.00	73.75	0.6429	13194	ok
isEmail	naive	whitebox_code_only	success	43/47	0.9149	66.67	52.63	100.00	67.50	0.6429	13120	ok
isEmail	improved	blackbox	success	40/43	0.9302	79.76	75.00	100.00	80.00	1.0000	14843	ok
isEmail	improved	whitebox	success	47/57	0.8246	94.05	90.79	100.00	93.75	0.9286	98631	ok
isEmail	improved	whitebox_code_only	success	36/47	0.7660	97.62	92.11	100.00	97.50	0.9286	50120	ok
isURL	naive	blackbox	success	44/50	0.8800	64.41	52.10	100.00	65.79	0.8182	14416	ok
isURL	naive	whitebox	success	52/64	0.8125	66.10	60.50	100.00	67.54	0.8182	18662	ok
isURL	naive	whitebox_code_only	success	35/41	0.8537	72.88	63.87	100.00	74.56	0.5455	15721	ok
isURL	improved	blackbox	success	34/41	0.8293	64.41	54.62	100.00	65.79	1.0000	55039	ok
isURL	improved	whitebox	success	54/64	0.8438	96.61	86.55	100.00	96.49	1.0000	76002	ok
isURL	improved	whitebox_code_only	success	36/39	0.9231	78.81	66.39	100.00	78.07	0.7273	146922	ok
isFQDN	naive	blackbox	success	43/48	0.8958	85.29	84.21	100.00	84.85	0.8333	7897	ok
isFQDN	naive	whitebox	success	59/60	0.9833	91.18	89.47	100.00	90.91	0.8333	40602	ok
isFQDN	naive	whitebox_code_only	success	13/13	1.0000	91.18	92.11	100.00	90.91	0.1667	36714	ok
isFQDN	improved	blackbox	success	28/30	0.9333	82.35	81.58	100.00	81.82	0.8333	8538	ok
isFQDN	improved	whitebox	success	27/35	0.7714	85.29	84.21	100.00	84.85	1.0000	65143	ok
isFQDN	improved	whitebox_code_only	success	22/26	0.8462	91.18	92.11	100.00	90.91	1.0000	26464	ok
isCurrency	naive	blackbox	success	48/52	0.9231	85.37	75.00	100.00	84.21	0.3750	13756	ok
isCurrency	naive	whitebox	success	74/82	0.9024	90.24	80.56	100.00	89.47	0.5625	16352	ok
isCurrency	naive	whitebox_code_only	success	72/96	0.7500	87.80	72.22	100.00	89.47	0.8125	20465	ok
isCurrency	improved	blackbox	success	52/62	0.8387	97.56	97.22	100.00	97.37	1.0000	20067	ok
isCurrency	improved	whitebox	success	45/67	0.6716	100.00	100.00	100.00	100.00	1.0000	39595	ok
isCurrency	improved	whitebox_code_only	success	35/49	0.7143	100.00	100.00	100.00	100.00	1.0000	23228	ok
isCreditCard	naive	blackbox	success	25/33	0.7576	100.00	94.74	100.00	100.00	0.4444	8137	ok
isCreditCard	naive	whitebox	success	20/22	0.9091	100.00	94.74	100.00	100.00	1.0000	8662	ok
isCreditCard	naive	whitebox_code_only	success	10/11	0.9091	100.00	94.74	100.00	100.00	0.3333	6999	ok
isCreditCard	improved	blackbox	success	22/25	0.8800	95.83	84.21	100.00	95.45	1.0000	9199	ok
isCreditCard	improved	whitebox	success	21/22	0.9545	100.00	100.00	100.00	100.00	1.0000	11953	ok
isCreditCard	improved	whitebox_code_only	success	11/17	0.6471	83.33	57.89	100.00	81.82	1.0000	22939	ok

7. 详细对比分析

本节分两个维度来分析：

1. 同一 mode 下，`improved` 相对 baseline `naive` 提升了什么；
2. 同一 agent 内部，不同 mode 带来了什么信息增益或风险。

7.1 同一 mode 下：`naive` baseline vs `improved`

7.1.1 `blackbox`

- `naive blackbox`：`Exact 0.8826`，`Golden 0.5942`，`Tokens 11259.80`
- `improved blackbox`：`Exact 0.8823`，`Golden 0.9667`，`Tokens 21537.20`

结论：

- correctness 几乎持平；
- `improved` 的黄金命中率大幅提升；
- 成本约增加一倍。

原因：

- `naive blackbox` 只是一个保守 baseline；
- `improved blackbox` 使用 obligations extraction、category clustering 与 self-audit；
- 因此更容易覆盖 requirement 中分散的语义类别和 option family。

这说明在 blackbox 场景下，`improved` 的主要增益首先来自**prompt 增强**，而不是 iteration。

7.1.2 `whitebox`

- `naive whitebox`：`Exact 0.9167`，`Stmt 84.27`，`Branch 78.74`，`Line 84.33`，`Golden 0.7714`，`Tokens 19494.40`
- `improved whitebox`：`Exact 0.8132`，`Stmt 95.19`，`Branch 92.31`，`Line 95.02`，`Golden 0.9857`，`Tokens 58264.80`

结论：

- baseline `naive` 的 correctness 更高；
- `improved` 的 coverage 和 golden overlap 明显更高；
- `improved` 成本显著更高。

原因：

- `naive` 更保守，不主动追 coverage maximality，因此更不容易写歪 test case；
- `improved whitebox` 则结合了更强的 white-box prompt 与 coverage-guided patch 迭代；
- 因而更容易补到分支与语义类别，但也更容易在参数、断言和边界细节上出错。

这说明：
`improved whitebox` 的主要优势不是 correctness，而是**结构覆盖能力与语义覆盖能力**。

7.1.3 `whitebox_code_only`

- `naive whitebox_code_only`：`Exact 0.8855`，`Golden 0.5002`，`Tokens 18603.80`
- `improved whitebox_code_only`：`Exact 0.7793`，`Golden 0.9312`，`Tokens 53934.60`

结论：

- `improved` 的黄金命中与 coverage 更强；
- baseline `naive` 的 correctness 更稳；

- `improved` 成本很高。

原因：

- code-only 情况下没有 requirement 约束；
- `improved` 更积极地依据代码分支扩展测试类别；
- 但越激进地从代码推断外部行为，越容易出现“路径上像对了、语义上却不完全对”的问题。

7.2 同一 agent 内部：不同 mode 的比较

7.2.1 baseline `naive` 内部对比

- `blackbox`：`Exact 0.8826`
- `whitebox`：`Exact 0.9167`
- `whitebox_code_only`：`Exact 0.8855`

观察：

1. `naive whitebox` 是 baseline 中表现最好的模式。
2. `naive blackbox` 虽然 coverage 与 golden 不如 `whitebox`，但成本最低、correctness 也很稳。
3. `naive whitebox_code_only` correctness 仍然不错，但 golden overlap 最低。

解释：

- 对 baseline 来说，requirement + code 的组合提供了最平衡的信息；
- 只有 requirement 时，模型仍能写出较稳的一轮 baseline；
- 只有 code 时，模型容易抓到 obvious branches，却不一定能恢复出专家测试的语义结构。

7.2.2 `improved` 内部对比

- `blackbox`：`Exact 0.8823`，`Golden 0.9667`
- `whitebox`：`Exact 0.8132`，`Golden 0.9857`，`Stmt 95.19`，`Branch 92.31`
- `whitebox_code_only`：`Exact 0.7793`，`Golden 0.9312`

观察：

1. `improved blackbox` 是最平衡的模式。
2. `improved whitebox` 是最强的 coverage 模式。
3. `improved whitebox_code_only` 风险最大。

解释：

- `improved blackbox` 主要依靠 prompt 增强，就已经能显著提升语义类别覆盖；
- `improved whitebox` 则进一步利用代码结构与 uncovered feedback，把路径探索能力拉到最高；
- `improved whitebox_code_only` 虽然也能追路径，但由于 requirement 缺失，更容易语义漂移。

8. 消融实验

本节重点回答两个问题：

1. `improved whitebox` 要不要加 requirement 复检?
2. 为什么 `suite + patch` 比纯 suite 重写更好?

为了公平比较，本节不直接对比不同轮次的整张总表，而是基于 `experiment_ablation_1.md`、`experiment_ablation_2.md` 与 `experiment_final_1.md`，统一采用相同的 `5` 个函数口径：

- `isEmail`
- `isURL`
- `isFQDN`
- `isCurrency`
- `isCreditCard`

然后只比较 `improved whitebox` 的均值。

8.1 复检机制的消融：要不要加 requirement 复检

`experiment_ablation_1.md` 的 `improved whitebox` 对应“加入 requirement 复检 / pruning”的版本；最终版则去掉了这一机制。

同口径 `5` 函数下，`improved whitebox` 对比如下：

Version	Avg Tokens	Avg Exact Match	Avg Stmt Cov	Avg Branch Cov	Avg Func Cov	Avg Line Cov	Avg Golden Match
`experiment_ablation_1` (带 requirement 复检)	79361.20	0.8849	88.38	82.59	100.00	88.49	0.7776
`final` (最终版)	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857

这个结果说明：

1. requirement 复检确实能在这组数据里保住更高 correctness；
2. 但代价是：
    - token 更高；
    - coverage 更低；
    - golden overlap 显著更低。

因此，对 `improved whitebox` 来说，复检机制并没有把它变成“更平衡”的方法，反而削弱了它原本最有价值的能力：

- 路径探索；
- 语义类别扩展。

项目最终判断：
requirement 复检是错误方向。

8.2 迭代方式的消融：为什么 `suite + patch` 比纯 suite 重写更好

`experiment_ablation_2.md` 对应的是删除复检机制后的版本；最终版则进一步收敛到当前的 `suite + patch` 白盒迭代方式。

同口径 `5` 函数下，`improved whitebox` 对比如下：

Version	Avg Tokens	Avg Exact Match	Avg Stmt Cov	Avg Branch Cov	Avg Func Cov	Avg Line Cov	Avg Golden Match
`experiment_ablation_2`	68342.80	0.8164	88.74	81.49	100.00	89.37	0.8430
`final` (suite + patch)	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857

这个结果说明：

1. `suite + patch` 能显著降低 token 成本；
2. `suite + patch` 能明显提升 coverage；
3. `suite + patch` 能明显提升 golden overlap；
4. correctness 基本持平，只是略微下降。

因此，更准确的结论不是“`suite + patch` 大幅提高 correctness”，而是：

在几乎不损伤 correctness 的前提下，`suite + patch` 显著提升了 coverage / semantic overlap，并降低了成本。

9. 项目实践经验

本节不重复消融实验本身，而总结在整个研发和实验过程中得到的更一般性的经验。

9.1 太简单的函数必须剔除

太简单的函数会让所有方法都表现很好，从而稀释困难函数上的真实差异。
因此，benchmark 设计本身就是实验可信度的重要组成部分。

9.2 高 coverage 不等于高质量测试

在 `isCurrency` 等案例上，多次出现：

- coverage 很高；
- golden overlap 很高；
- 但 correctness 偏低。

这说明 AI 生成测试时，不能只追 coverage。
如果没有 correctness 约束，高覆盖率可能只是“走到了很多路径”，并不意味着写对了测试。

9.3 code-only 模式天生更危险

只给代码时，模型更容易学到实现结构，却更容易丢失 requirement 对输入语义、边界和外部行为的约束。
因此 `whitebox_code_only` 更适合作为补充视角，而不是默认主力模式。

9.4 baseline 非常重要

如果没有 `naive baseline`，很多复杂设计看起来都会“很先进”。
但一旦与一个稳定、保守、低风险的 baseline 对比，就会发现：

- 有些复杂设计只是提高了 token 成本；
- 有些复杂设计提高了 coverage，却没有提高 correctness；
- 只有少数设计真正值得保留。

因此，baseline 在本项目里不是无关紧要的配角，而是验证研发改进是否真的有效的关键参照。

10. 与人类专家修复的对比

为了完整评估该方法的定位，还需要与传统 non-AI 方法进行比较。
本项目没有实现一个完整的自动传统测试生成器，但可以清晰比较 AI 方法与人工/专家修复方式的优劣。

10.1 人类专家修复的优点

- correctness 更稳定；
- requirement 误读更少；
- 更容易做关键边界条件确认；
- 更适合作为高风险 case 的最终把关。

10.2 AI 方法的优点

- 生成速度快；
- 能快速扩展候选测试与语义类别；
- 在白盒场景下有较强的路径探索能力；
- 很适合做初始测试草案与覆盖率探索。

10.3 AI 方法的不足

- 会误解 requirement；
- 会写错参数结构或断言；
- coverage 高不代表 correctness 高；
- 没有 requirement 时更容易语义漂移。

10.4 更现实的定位

因此，本项目认为 AI 更适合作为：

测试设计加速器 / 草案生成器 / 覆盖率探索工具

而不是完全替代人工测试工程师。

一个更现实的工作流是：

1. 先由 AI 生成候选测试集；
2. 再由人工或规则化检查做筛选、修正和最终确认。

11. 方法的泛化能力

本项目关注 generalization，而不是只看单个函数是否“碰巧成功”。

11.1 跨函数类型的泛化

最终纳入的 `5` 个函数并不属于同一种简单模式：

- `isEmail`：规则复杂、选项多；
- `isURL`：路径与选项组合复杂；
- `isFQDN`：结构清晰但边界较多；
- `isCurrency`：格式与选项耦合明显；
- `isCreditCard`：校验规则较强、边界较敏感。

尽管函数风格不同，实验中仍然能反复看到稳定模式：

- baseline `naive` 更稳；
- `improved` 更强 coverage / golden；
- code-only 风险更高；
- `suite + patch` 优于整套重写。

这说明结论不是只在某一个函数上成立，而是在多个不同 validator 上都可重复观察到。

11.2 跨输入模式的泛化

本项目同时覆盖：

- 只给 requirement；
- requirement + code；
- 只给 code。

因此，方法的泛化性不仅体现在“跨函数”，也体现在“跨信息条件”。

实验表明：

- 当 requirement 可得时，模型更容易保持 correctness；
- 当 code 可得时，模型更容易提升 coverage；
- 当只有 code 时，模型更容易语义漂移。

这说明结论对不同测试输入场景都有解释力。

11.3 跨设计版本的泛化

本项目经历了多轮设计演化：

- 带 requirement 复检；
- 去掉 requirement 复检；
- 纯 suite 重写；
- `suite + patch` 迭代。

在这些设计版本变化中，一些现象始终稳定存在：

- `improved whitebox` 的核心优势是 coverage / golden，而不是 correctness；
- baseline `naive` 的核心优势是 correctness 稳定性；
- 仅代码模式更不稳定。

这说明项目结论并不是偶然来自某一次实验，而是具有一定跨版本稳定性。

11.4 泛化性的边界

当然，本项目的泛化性仍有边界：

- 当前实验对象主要集中在 `validator.js` 风格函数；
- 尚未扩展到更大型、跨模块、状态更复杂的系统级软件；
- 因此不能直接声称该方法已泛化到所有软件测试场景。

更谨慎的结论是：

本方法已经在真实开源库中的 validator-style functions 上表现出较好的跨函数、跨模式、跨版本稳定性，但更复杂软件系统的泛化仍需后续验证。

12. 项目中各产物的位置

围绕本项目的方法实现与实验分析，已经形成以下主要产物：

1. `Input`
   - requirement 与 project code base，来自 `validator.js` 文档与源码。
2. `Tool artifact`
   - prompts、模型调用逻辑、模型生成 JSON 测试集，位于 `agent_toolkit` 相关目录中。
3. `Generated output`
   - 各 mode / agent 的测试生成结果，位于 `agent_toolkit/outputs`。
4. `Experimental analysis`
   - 包括 correctness、coverage、golden overlap、token cost 与多轮 ablation，见各 `experiment_*.md`。
5. `Project report`
   - 本文档为 `report_1.md`；原始分析材料见各实验报告。

本项目使用的模型为实验日志中记录的 `gpt-4o`，并记录 token usage 以支持成本分析。

13. 总结

本项目在真实开源函数库 `validator.js` 上实现并评估了一个 LLM 驱动的测试生成工具，并围绕 `improved agent` 展开了系统性的设计与验证。

最终可以得出以下结论：

1. LLM 的确可以生成真实可执行测试，但测试质量必须用 correctness、coverage、golden overlap 联合评估。
2. `naive agent` 作为 baseline，提供了一个稳定、保守、偏 correctness 的参考上界。
3. `improved agent` 的核心贡献主要来自两部分：更强的 prompt 设计，以及覆盖率驱动的白盒 patch 迭代。

4. `improved whitebox` 是 coverage 与 golden overlap 最强的方案，但 correctness 仍未追平 baseline `naive whitebox`。
5. `improved blackbox` 是一个非常平衡的方案，说明 prompt 增强本身就能显著改善 black-box 测试生成。
6. 对 `improved whitebox` 而言，requirement 复检是错误方向；`suite + patch` 才是更有效的迭代策略。
7. 本方法已经在多个 validator-style functions、多个输入模式和多个设计版本上表现出一定泛化性，但距离更复杂系统级软件测试的全面泛化仍有距离。

因此，本项目的总体结论不是“AI 已经能够取代人工测试设计”，而是：

AI 最适合作为测试设计加速器与覆盖率探索工具；而高可靠测试质量，仍然需要更稳健的约束机制，甚至需要人工参与修正。

附录 A：本项目使用的主要 Prompt

下面展示本项目中最核心的 prompt 模板。为方便阅读，使用占位符表示动态插入内容。

A.1 `naive` 系统 Prompt

```
Generate a conservative first-pass validator.js-style test suite.
Prefer obvious behaviors, obvious edge cases, and small grouped coverage over
broad exploration.
Keep the suite intentionally naive: do not optimize for completeness, novelty, or
deep branch hunting.

Always return valid JSON only.
JSON schema:
{
  "obligations": [],
  "test_groups": [
    {
      "title": "string",
      "validator": "target validator name",
      "args": [ { } ],
      "valid": ["..."],
      "invalid": ["..."],
      "rationale": "string",
      "obligations": ["OB-1", "OB-2"]
    }
  ]
}

Requirements:
- Group tests by option profile.
- obligations may be an empty list.
- Prefer a small number of clear groups over many speculative groups.
- Prefer common cases and easy-to-justify boundaries.
- Avoid complex combinations unless they are directly implied by the requirement.
- Never repeat the same JSON key within one object.
- Keep the response concise enough to remain valid JSON.
- Do not output markdown.
```

A.2 `naive blackbox` Prompt

```
Task: generate black-box tests for {validator_name}.

Read the requirement and directly produce a small first-pass grouped
validator.js-style test suite.
Stay conservative and only cover behaviors that are explicit or strongly implied
by the requirement.
Do not spend effort extracting formal obligations.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Black-box guidance:
- prefer common positive and negative examples
- use only a few clearly differentiated groups
- avoid speculative option interactions
- stop at a modest baseline instead of trying to be comprehensive

Requirement specification:
{requirement_spec}
```

A.3 `naive whitebox` Prompt

```
Task: generate white-box tests for {validator_name}.

Read the requirement and source code, then directly produce a small first-pass
grouped validator.js-style test suite.
Use the source code only to notice obvious branches or guards, not to
aggressively chase coverage.
Do not try to build a detailed obligation hierarchy.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Requirement specification:
{requirement_spec}

Source code:
{source_code}

Output guidance:
- keep the suite modest in size
- each group should focus on one option profile or one branch family
- prefer obvious branches and obvious edge cases over exhaustive exploration
- avoid patch-like additions or coverage-maximizing behavior
- keep the result as a conservative baseline, not an ambitious suite
```

A.4 `naive whitebox_code_only` Prompt

```
Task: generate white-box tests for {validator_name} using source code only.

Read the source code and directly produce a small first-pass grouped
validator.js-style test suite.
There is no external requirement specification in this mode.
```

```
Infer externally visible behavior conservatively from obvious guards, branches,
and option handling in the code.
Do not try to build a detailed obligation hierarchy.
Use an empty obligations list unless there is a very obvious single high-level
rule to note.

Source code:
{source_code}

Output guidance:
- keep the suite modest in size
- each group should focus on one option profile or one obvious branch family
- prefer obvious branches, guards, and obvious edge cases over exhaustive
exploration
- avoid speculative behavioral claims that are not well supported by the code
- keep the result as a conservative baseline, not an ambitious suite
```

A.5 `improved` 系统 Prompt

```
You are an expert software testing agent.
You generate high-quality black-box and white-box tests for validator-style
functions.
You must reason conservatively, preserve correctness, and improve coverage with
small precise patches.

Always return valid JSON only.
JSON schema:
{
  "obligations": [
    {
      "id": "string",
      "kind": "blackbox|whitebox",
      "rule": "string",
      "why": "string",
      "minimal_valid": "string or null",
      "minimal_invalid": "string or null"
    }
  ],
  "test_groups": [
    {
      "title": "string",
      "validator": "target validator name",
      "args": [ { } ],
      "valid": ["..."],
      "invalid": ["..."],
      "rationale": "string",
      "obligations": ["OB-1", "OB-2"]
    }
  ]
}

Requirements:
- Group tests by option profile.
```

```
- For broad black-box APIs with many documented options, expand into multiple
semantically distinct groups instead of collapsing everything into one default
group.
- Keep each group narrow: a group should usually cover one baseline partition
family or one closely related option-sensitive behavior.
- Prefer more distinct categories when the requirement clearly documents many
switches, constraints, or acceptance/rejection modes.
- Keep patches minimal and avoid redundant duplicates.
- Prefer minimally different cases that flip one rule at a time.
- Never repeat the same JSON key within one object.
- Keep the response concise enough to remain valid JSON.
- Do not output markdown.
```

A.6 `improved blackbox` Prompt

```
Task: generate black-box tests for {validator_name}.

Apply standard black-box testing methods aggressively:
- equivalence partitioning
- boundary value analysis
- option-combination testing
- negative-input testing
- special-case and rare-option exploration

Step 1: extract explicit black-box obligations from the requirement.
Step 2: cluster the obligations into semantically distinct black-box categories.
Step 3: expand each category into grouped validator.js-style tests.
Step 4: self-check whether rare options, boundary-adjacent cases, invalid
partitions, and documented option flips were skipped.

Requirement specification:
{requirement_spec}

Output guidance:
- first identify the major requirement-visible categories before writing any
examples
- keep one coherent option profile or one closely related rule family per group
- do not collapse many unrelated options into one catch-all "default" group
- if the requirement documents many independent switches or acceptance/rejection
rules, reflect them in multiple groups
- cover both baseline behavior and option-sensitive deviations
- prefer broad category exploration, but keep each example requirement-grounded
and likely correct
- for every option-sensitive group, choose examples whose validity actually
changes because of that option profile
- if a documented option effect is only observable together with another option
value, configure both so the distinction is real
- do not claim an example is invalid for an option if the requirement text still
permits it under the provided args
- prefer paired examples that differ by one rule or one option flip when
demonstrating requirement changes
- when a default already allows a behavior, do not present enabling the same
behavior as a meaningful new category unless another argument makes the contrast
observable
```

```
- when uncertain, choose fewer examples inside a group rather than merging
unrelated categories
- include multiple concrete valid and invalid examples only when they cover
meaningfully different cases
- try to ensure most obligations are represented by at least one dedicated or
clearly focused group

Self-audit before finalizing:
- Did you create separate groups for materially different option families instead
of one oversized group?
- Did you include default behavior, strictness-raising options, permissive
options, whitelist/blacklist style filters, and length-related behavior when
documented?
- For each option-focused group, would at least one valid/invalid pair change
outcome if that option were flipped back?
- Did you accidentally use examples whose expected result depends on undocumented
implementation details rather than the visible requirement?
- Are your expectations aligned with the requirement text rather than guessed
from hidden implementation details?
```

A.7 `improved whitebox` Prompt

```
Task: generate white-box tests for {validator_name}.

Treat this as requirement-constrained white-box testing.
Use both the intended external behavior and the implementation structure.

Apply standard white-box testing methods aggressively:
- statement coverage
- branch coverage
- condition-oriented reasoning
- early-return triggering
- helper-sensitive path exploration
- requirement-grounded partition checking

Step 1: derive black-box obligations from the requirement.
Step 2: enumerate structural white-box obligations directly from the code.
Step 3: merge overlaps and contradictions.
Step 4: expand them into many grouped validator.js-style tests.
Step 5: self-check whether early returns, helper-dependent checks, option flips,
boundary-triggered branches, and requirement-visible edge cases were skipped.

Requirement specification:
{requirement_spec}

Source code:
{source_code}

Output limits:
- each group should focus on one option profile or one branch family
- explore broadly when requirement-visible behavior and source-level branches
genuinely justify it
- do not inflate obligations or groups just to reach a target count
```

```
- prefer a compact suite that still covers materially different paths and edge
cases
```

A.8 `improved whitebox_code_only` Prompt

```
Task: generate white-box tests for {validator_name} using source code only.

Treat this as code-only white-box testing.
There is no external requirement specification in this mode.
Use the implementation structure to infer externally visible behavior
conservatively.

Apply standard white-box testing methods aggressively:
- statement coverage
- branch coverage
- condition-oriented reasoning
- early-return triggering
- helper-sensitive path exploration
- conservative behavior inference from code-visible contracts

Step 1: enumerate structural white-box obligations directly from the code.
Step 2: infer only well-supported externally visible behaviors from those
structures.
Step 3: expand them into grouped validator.js-style tests.
Step 4: self-check whether early returns, helper-dependent checks, option flips,
and boundary-triggered branches were skipped.

Source code:
{source_code}

Output limits:
- each group should focus on one option profile or one branch family
- explore broadly when the source code clearly justifies it
- do not inflate obligations or groups just to reach a target count
- prefer a compact suite that still covers materially different paths and edge
cases
- do not make speculative claims about undocumented behavior unless the code
strongly supports them
```

A.9 `improved` 白盒迭代 Prompt（最终版）

```
You are improving an existing {mode} test suite for {validator_name}.
You are given only coverage details and fine-grained uncovered white-box
locations.
You must use those uncovered positions as exact white-box targets when updating
the suite.
You must NOT use any golden tests, repository test overlap analysis, or external
oracle hints.

Goal:
- propose a small patch that helps the current suite target more uncovered code
locations
```

```
- preserve the current suite unless a new patch is clearly justified by uncovered
locations
- keep the patch conservative and additive whenever possible
- keep the suite conservative and low-redundancy

Return strict JSON with top-level keys obligations and test_groups.
The returned test_groups must be patch groups only, not a full rewritten suite.

Black-box requirement source:
{requirement_spec_or_none}

Current full test suite from the previous iteration:
{generated_groups}

Coverage details allowed for improvement:
{evaluation_feedback}

Instructions:
- You must use only the requirement (if provided), the current suite, coverage
details, and uncovered details.
- Do not infer or use any correctness signal, pass/fail summary, failed cases, or
hidden oracle information.
- Treat uncovered_details as exact white-box targets, not as vague hints.
- Use the previous full suite as the baseline and return only the incremental
patch groups.
- Do not rewrite, delete, or duplicate the existing groups already shown in the
current suite.
- Add only groups that cover meaningfully new targets beyond the current suite.
- Prefer at most 3 patch groups, each focused on one concrete uncovered condition
or path family.
- If an uncovered branch or statement cannot be operationalized from the provided
coverage details, do not guess.
- If uncovered_details are provided, use them only as white-box targets.
- If uncovered_details are not provided, do not infer any hidden coverage gap.
- Prefer minimal option-profile-specific groups.
- Avoid broad extrapolation; every meaningful update should correspond to a
concrete uncovered position or condition.
- Do not mention or infer any golden test information.
```

A.10 JSON 修复 Prompt

```
Repair the following invalid JSON into strict valid JSON.
Keep the same schema with top-level keys obligations and test_groups.
Remove duplicated keys, remove obviously repeated corrupted fragments, and
preserve as much valid content as possible.
Return JSON only.

{raw_response}
```

A.11 Group Completion Prompt

```
The previous {mode} generation for {validator_name} produced obligations but no
usable test_groups.
Using the obligations below, generate non-empty validator.js-style test_groups
now.
Return strict JSON with top-level keys obligations and test_groups.
Keep the obligations unchanged unless a tiny correction is necessary.
test_groups must be non-empty.
Each group must contain validator, args, valid, invalid, title, rationale,
obligations.

Original task:
{original_prompt}

Existing obligations:
{obligations}
```